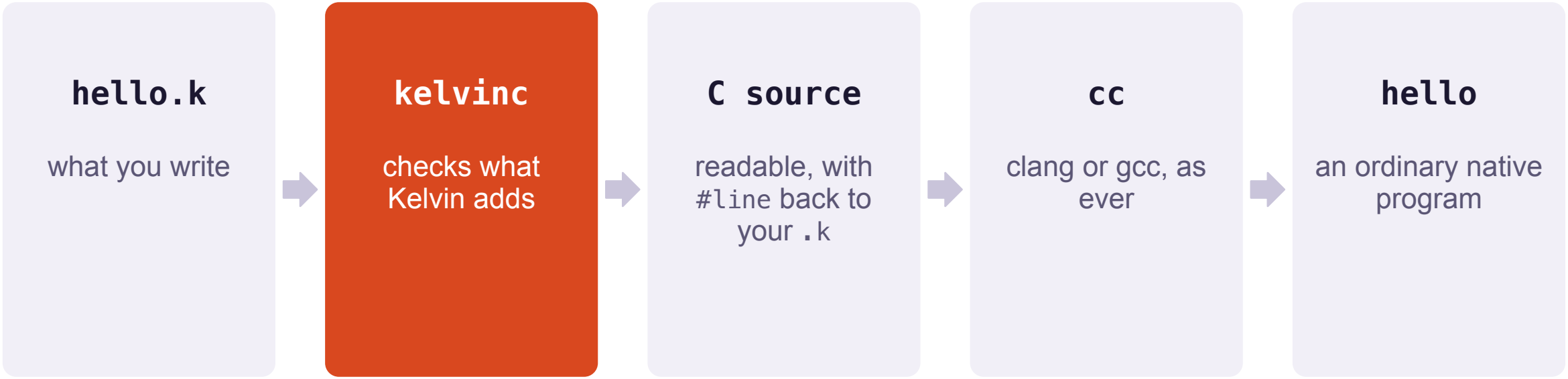
Kelvin

C, changed bit by bit.

A quiet alternative to C, still 100% ABI compatible.

Kelvin is C with different spelling

`kelvinc` translates Kelvin into C and hands that to your C compiler.



Types, layout, calling conventions and semantics are C's by construction.

The name is the design

Dan

Let's start implementing kelvin. An attempt to implement a `better C`. It is virtually named by you:

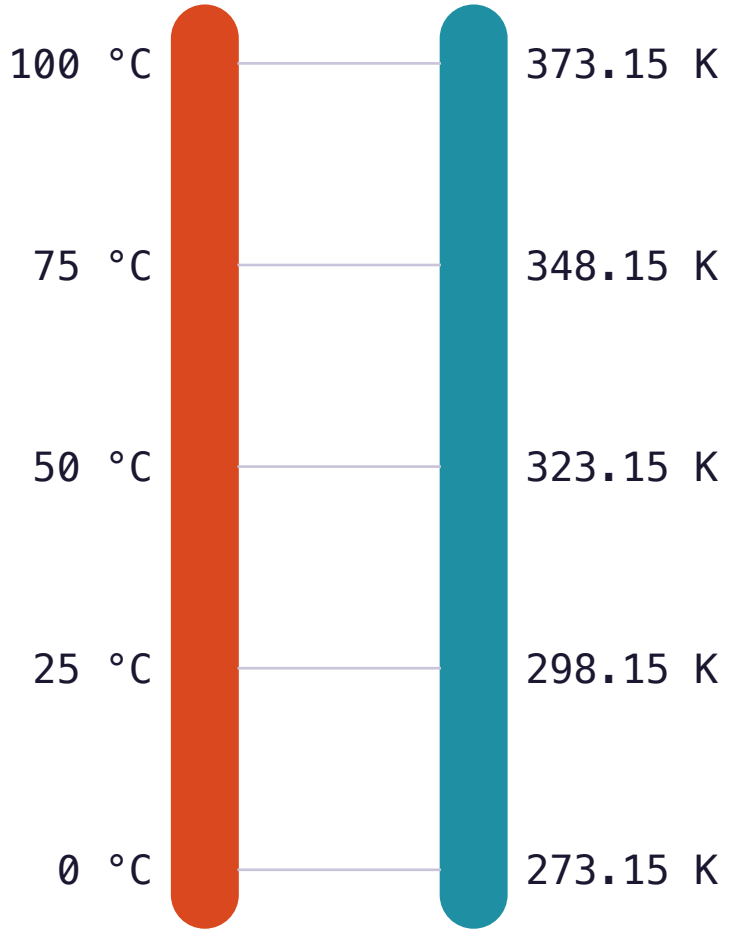
Claude

I'd call it **Kelvin**.

C is also the symbol for Celsius, and Kelvin is its natural successor. The step size is identical, so everything you know about C carries over one-to-one…

Kelvin wins because the pun is also a design spec.

Dialogue.md, sections 1 and 2. Runners-up: Sequel, P, Coda, Seam.



Declarations read left to right

C

```c
#include <stdint.h>

struct point { int64_t x, y; };

int64_t add(int64_t a, int64_t b) {
    return a + b;
}

int main(void) {
    const int64_t k = 7;
    int32_t n = 0;
    int32_t *p = &n;
    *p = 9;
    return n;
}
```

Kelvin

```
struct point { x: i64; y: i64 }

let add(a:i64, b:i64):i64 {
    return a + b
}

let main():i32 {
    let k = 7
    var n:i32 = 0
    var p:i32^ := &n
    p^ = 9
    return n
}
```

C stays C

fromc.k

```
#import <stdio.h> as C     // C's own header

let square(n:i32):i32 { return n * n }

let main():i32 {
    let k = 7              // an i64, and a let
    var total:i32 = 0
    var p:i32^ := &total   // C's int32_t *
    p^ = square(3)
    printf("%d %lld\n", total, k)
    return 0
}
```

Headers come in as they are

Precedence and conversions are C's

Undefined behavior stays, as written

prints

```
9 7
```

You can read the C it writes

sum.k

```
struct point { x: i64; y: i64 }

let sum(p:point):i64 {
    return p.x + p.y
}

let main():i32 {
    var total:i64 = 0
    for i in 0..<3 { total += i }
    return i32(total)
}
```

kelvinc --emit-c sum.k

```
struct point
{
    int64_t x;
    int64_t y;
};
int64_t sum(const struct point p)
{
    return p.x + p.y;
}
```

A `let` parameter is simply `const`. A Kelvin struct is a C struct, and a Kelvin function a C function.

Why C, in 2026?

C lets you shoot your own foot, and Uncle Sam tells you to avoid it.

The case against, all of it true

2024
the White House asks for memory safe languages, and names C and C++ as lacking it

2025
the NSA and CISA press the same case

Today
Go, Rust and Swift are memory safe, and emit native binaries

Kelvin stands on C anyway

1. **libc**, or libSystem on macOS, is there to stay
2. **The C compiler** is the most basic compiler infrastructure, on every platform we can think of
3. **Standing on C** makes Kelvin more than ten times smaller
4. **Memory safety** can be built within C's features, and can look like Rust or Swift

libc and cc are already there

```
otool -L of hello world, built on a Mac
Kelvin   /usr/lib/libSystem.B.dylib

Rust     /usr/lib/libSystem.B.dylib

Go       /usr/lib/libSystem.B.dylib
         /usr/lib/libresolv.9.dylib

Swift    /usr/lib/libSystem.B.dylib
         /usr/lib/swift/libswiftCore.dylib
```

Go 1.12's release notes: "libSystem is now used when making syscalls on Darwin."

libc stays: all four link it here, the memory safe three included

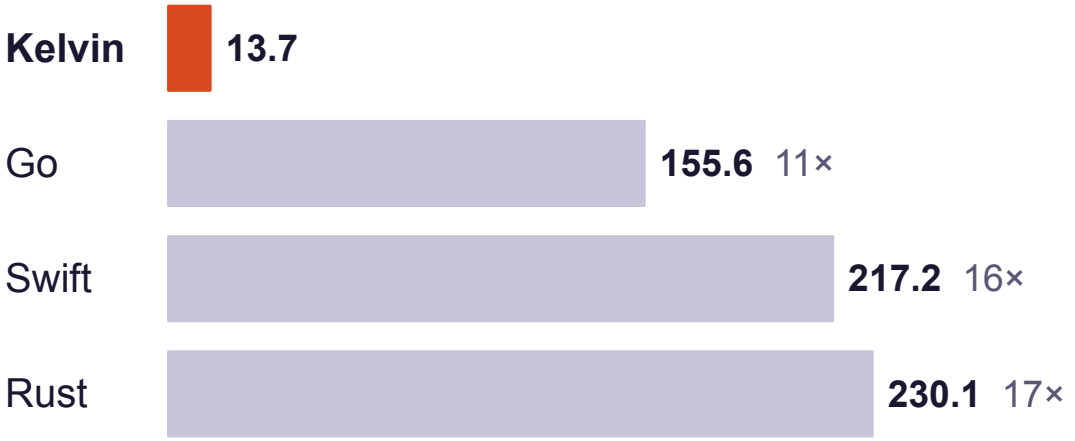
cc is everywhere: the most basic compiler infrastructure, on every platform we can think of

So Kelvin reuses both, and has neither to port

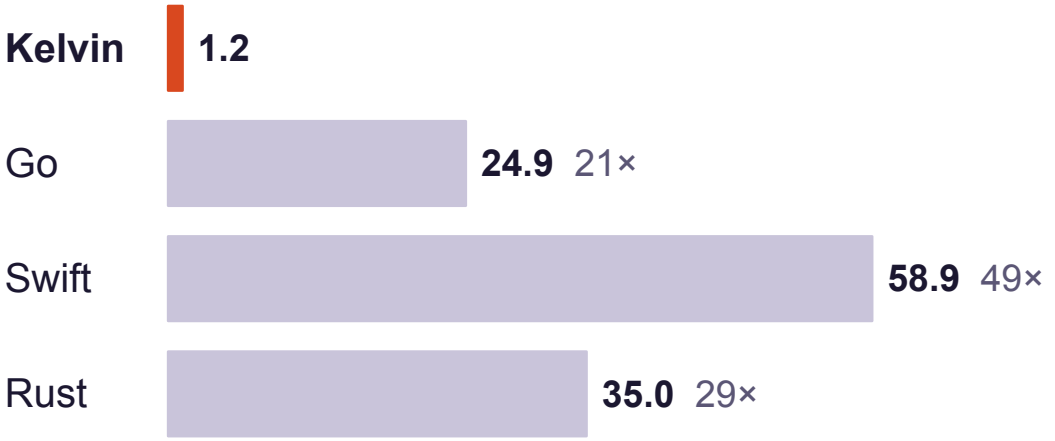
Standing on C keeps Kelvin small

More than ten times smaller, and more than half of it is Unicode data.

Every tracked file, MB



The compiler's own source, MB



What is in Kelvin's 13.7 MB



Measured 2026-11-07 with `intro/measure.sh`, on each default branch. LLVM and the C compiler are counted for no one, and the others do far more.

Looks like Rust or Swift, transpiles to C

safer.k

```
let half(n:i64):i64? {
    if n % 2 != 0 { return none }
    return n / 2
}

let main():i32 {
    let s = $"owned"
    let h = half(7)
    println(s, " ", h)
    return 0
}
```

kelvinc --emit-c safer.k

```
struct _kv_opt_i64 half(const int64_t n)
{
    if (n % 2 != 0)
    {
        return _kv__kv_opt_i64_make_none();
    }
    return _kv__kv_opt_i64_make_some(n / 2);
}

__attribute__((cleanup(kv_string_free)))
kv_string s = KV_STRING_LIT("owned");
const struct _kv_opt_i64 h = half(7);
```

Owners free themselves, `T?` must be unwrapped and `xs[i]` is checked, all in plain C. `^` pointers and C's undefined behavior stay, so Kelvin is safer C, without Rust's guarantee.

One $, and both worlds coexist

C's own literals stay exactly as they are. A single $ puts a flexible one beside each.

C's own: rigid, memory unsafe			With a $: flexible, memory safe
`"text"`	a `cstr`: a bare pointer to fixed bytes	$	`$"text"` a `String`: owns its bytes, grows
`[1, 2, 3]`	a fixed array: nothing checks a `[7]`	$	`$[1, 2, 3]` an `Array`: grows, and `xs[7]` stops the program
`{1, 2}`	a struct's fields, in order	$	`${"ann": 31}` a `Dictionary`, and `${1, 2}` is a `Set`
`// text`	a comment: C has no such literal	$	`$/k.l/i` a `RegExp`, its pattern checked as you compile

Short of a pointer taken into one, with `:=`, the $ side cannot be misused as C's can.

"as a human I feel the need for the shortcut." Dan, on the day $ arrived

Pointers got a little harder, on purpose

C: the classic, once admired

```c
#include <stdio.h>

int main(void) {
    char a[4] = "abc", b[4];
    char *p = b;
    const char *q = a;
    while ((*p++ = *q++) != '\0')
        ;
    puts(b);
    return 0;
}
```

Kelvin: one step a line

```
let main():i32 {
    var a:[u8](4) = "abc"
    var b:[u8](4)
    var p:u8^ := &b[0]      // := for a pointer
    var q:u8^ := &a[0]
    while q^ != 0 {
        p^ = q^
        p++                 // a statement
        q++
    }
    p^ = 0
    println(&b[0] as cstr)
    return 0
}
```

```
while (p++^ = q++^) != 0 { }
```

```
error: '++' is a statement in Kelvin, not a value: step the pointer on a line of its own
```

```
p = q
```

```
error: 'p' is a reference (i32^): assign it with ':=', not '='
```

Owners free themselves

owners.k

```
let shout(name:cstr):String {
    var s = String(name)   // owns its bytes
    s += "!"
    return s               // moved to the caller
}

let main():i32 {
    let hi = shout("hello")
    var xs = $[3, 1, 2]    // an Array<i64>
    xs += 4
    println(hi, " ", xs, " ", xs.count)
    return 0
}                          // hi and xs freed here
```

A block frees what its variables own

`return` **and passing** move an owner

`=` **never copies**: write `.copy()`, or borrow with `&`

prints

```
hello! [3, 1, 2, 4] 4
```

Mistakes are told in Kelvin's words




```
int x = 3
```

```
error: 'int' is not a Kelvin type; use i32 or u32
```

```
let n = 0
n += 1
```

```
error: 'n' is a let and cannot change; declare it with var
```

```
let a = $"x"
take(a)
println(a)
```

```
error: 'a' was moved out and holds nothing now: assign it
anew before using it
```

```
let m:i32? = 3
println(m + 1)
```

```
error: i32? has no operator '+': compare with == and !=
```

Failure is a value

result.k

```
#import <stdio.h> as C
#import <errno.h> as C

let open(path:cstr):Result<FILE^, i32> {
    let f:FILE^ := fopen(path, "r")
    if f.isNull { return err(errno) }
    return f                          // a T becomes ok(T)
}

let check(path:cstr):Result<i64, i32> {
    let f := try open(path)           // an err goes up
    fclose(f)
    return 1
}

let main():i32 {
    println(check("/no/such/file"))
    let first:i64? = none
    println(first ?? 42)
    return 0
}
```

`T?` and `Result<T, E>` are C structs underneath

`try` passes an error up

`??` gives a fallback

prints

```
err(2)
42
```

Any type can grow methods and operators

money.k

```
struct money { cents: i64 }

extend money {
    static let +(a:Self, b:Self):Self { (Self){a.cents + b.cents} }
    static let <(a:Self, b:Self):bool { a.cents < b.cents }
    let dollars:f64 { .cents / 100.0 }
    let doubled():Self { self + self }
}

let main():i32 {
    let a:money = {150}
    let b:money = {275}
    var bills = $[b, a]
    bills.sort()                // by its own <
    println(bills)
    println((a + b).cents, " ", a.doubled().dollars)
    return 0
}
```

Methods and attributes in `extend T { }`

Operators are functions chosen by their types

In C they are plain functions

prints

```
[{cents: 150}, {cents: 275}]
425 3.0
```

Collections and text are built in

values.k

```
struct point { x: i64; y: i64 }

let main():i32 {
    let squares = (1...5).map { $0 * $0 }
    let even = squares.filter { $0 % 2 == 0 }
    let ages = ${"ann": 31, "bob": 42}
    let name = $"ann"
    println(squares, " ", even)
    println(squares.sorted { $0 > $1 })
    println(ages)
    println((point){3, 4})
    println(`${name} is ${ages["ann"]!}`)
    return 0
}
```

`$[...]` and `${...}` make Arrays, Dictionaries and Sets

`{ $0 * $0 }` is an anonymous function, not a closure

`println` shows any value

prints

```
[1, 4, 9, 16, 25] [4, 16]
[25, 16, 9, 4, 1]
{"ann": 31, "bob": 42}
{x: 3, y: 4}
ann is 31
```

async and await are threads

tasks.k

```
let count(lo:i64, hi:i64):i64 {
    var n:i64 = 0
    for i in lo..<hi { n += i % 3 == 0 ? 1 : 0 }
    return n
}
let label(n:i64):String { $`${n} of them` }

let main():i32 {
    var parts:$[Task<i64>] = $[]
    for q in 0..<4 {
        parts += async count(q * 1000, (q + 1) * 1000)
    }
    var total:i64 = 0
    for i in 0..<parts.count { total += await parts[i] }
    let text = await async label(total)
    println(text)
    return 0
}
```

`async f(x)` runs `f` on a thread of its own

`await` waits, and gives its value

An owner moves back to the caller

prints

```
1334 of them
```

C calls Kelvin too

geo.k

```
struct point { x: f64; y: f64 }

let dist2(a:point, b:point):f64 {
    let dx:f64 = a.x - b.x
    let dy:f64 = a.y - b.y
    return dx * dx + dy * dy
}

// kelvinc --lib geo.k
// builds geo.kh, libkgeo.a and
// libkgeo.dylib or libkgeo.so
```

main.c

```
#include <stdio.h>
struct point { double x, y; };
double dist2__point__point(struct point a,
                           struct point b);

int main(void) {
    struct point a = {0, 0}, b = {3, 4};
    printf("%.1f\n",
           dist2__point__point(a, b));
    return 0;
}
```

From Kelvin it is `#import <geo>`. From C it is a library like any other.

prints

```
25.0
```

A library, one import at a time

`IO`
files, pipes and processes

`OS and SYS`
directories, environment, processes

`JSON`
`JSON.parse` and `JSON.stringify`, as JS has them

`SION`
SION values, JSON with more

`MsgPack`
MessagePack of a SION

`RegExp`
`$/pattern/flags`, JS's regular expressions

`UnicodeData`
normalization, graphemes, case mappings

`math`
`math.h` for floats and complex numbers

`Your own`
`kelvinc --lib M.k` builds one

Each comes in with `#import <M>`. What you do not import stays out.

Made one agreed change at a time

472
numbered changes

1,205
tests, with clang and with gcc 15

9
CI jobs on macOS, Ubuntu and FreeBSD

"Hey, you are going too fast!"

The first day's compiler added traps, slices and `defer` in one step. It was thrown away for bare C, and every departure since was proposed and agreed alone.

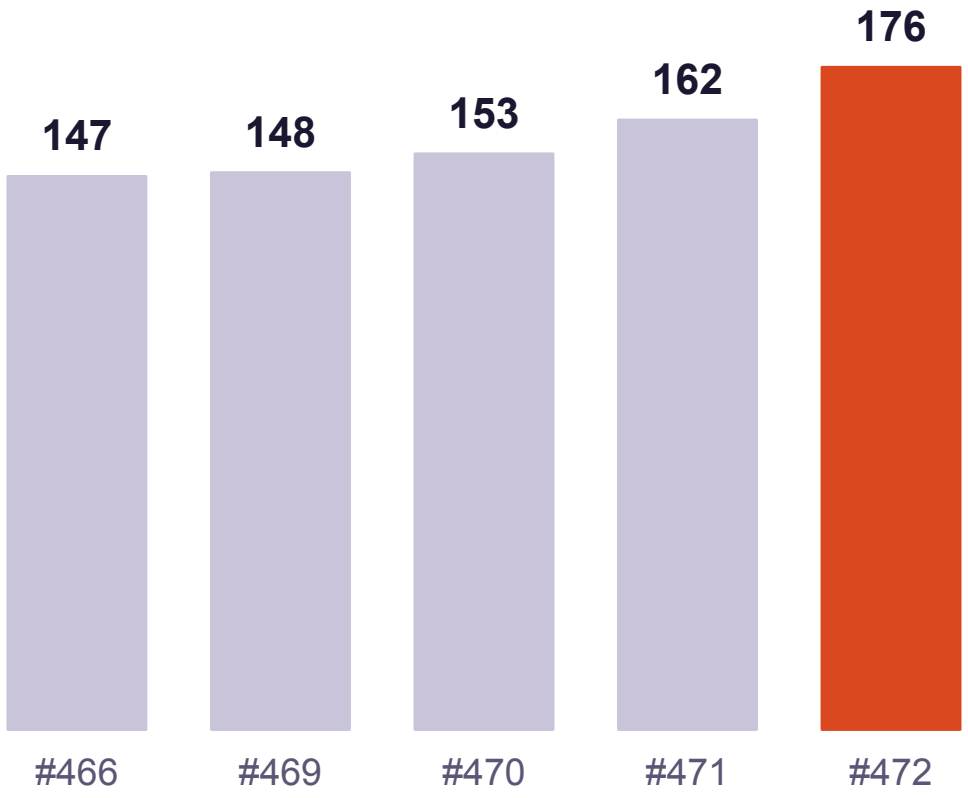
Design.md records what was decided

Dialogue.md records how, in the words used

Kelvin is learning to compile itself

`kc` is a compiler written in Kelvin. It builds itself to a fixpoint, in CI on every push.

Programs kc builds, by numbered change



Today	176 of 354 test programs and examples
Next	as capable as `kelvinc`
Then	skip C, from `kc` to assembly or LLVM IR
Someday	the whole backend in Kelvin

Try it

```
git clone https://github.com/dankogai/kelvin
cd kelvin
make
./kelvinc --run examples/hello.k
./kelvinc --emit-c examples/hello.k
make install PREFIX=~/.local
```

Needs: a C11 compiler, GNU make, Python 3

Runs on: macOS, Linux, FreeBSD; not Windows

Status: 0.1.0, so a later 0.x may change things

To read first: `documents/versusC.md`

In VS Code: `make vscode`

License: MIT

Everything you know about C carries over

github.com/dankogai/kelvin

MIT License